

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи № 8
з курсу “Операційні системи реального часу”
на тему: “Управління багатозадачною системою за допомогою ІЧ-
пульта”

Виконав:
Студент. гр. ФеП-41 Фіняк Олег
Перевірив:
Павлик М.Р.

Мета роботи. Навчитися приймати та обробляти сигнали від інфрачервоного пульта дистанційного керування та реалізувати вибір задач або режимів роботи системи за допомогою ІЧ-пульта.

```
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/adc.h"
#include "hardware/pio.h"

#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "semphr.h"

#include "DEV_Config.h"
#include "OLED_1in5.h"
#include "GUI_Paint.h"

#include "SHTC3.h"
#include "SGP40.h"
#include "QMI8658.h"
#include "nec_receive.h"

// ----- CONFIG -----
#define PIN_BUTTON 14
#define PIN_IR 15
#define PIO_BLOCK pio0

// ----- STRUCTS -----
typedef struct { float t, h; } TH_t;
typedef struct { uint32_t voc; } VOC_t;
typedef struct { float acc[3], gyro[3]; } IMU_t;
typedef struct { uint16_t value; } ADC_t;

// ----- GLOBALS -----
SemaphoreHandle_t i2cMutex;

QueueHandle_t qButton;
QueueHandle_t qTH;
QueueHandle_t qVOC;
QueueHandle_t qIMU;
QueueHandle_t qPot;
QueueHandle_t qLight;
QueueHandle_t qSound;
QueueHandle_t qIR;

bool display_on = true;
uint8_t mode = 0; // вибір відображення через ІЧ-пульт

// ----- BUTTON TASK -----
void vButtonTask(void *p){
    gpio_init(PIN_BUTTON);
    gpio_set_dir(PIN_BUTTON, GPIO_IN);
    gpio_pull_up(PIN_BUTTON);

    bool last = true, stable = true;
    int deb = 0;
```

```

while(1){
    bool r = gpio_get(PIN_BUTTON);
    if(r != last){
        deb = 3;
        last = r;
    } else if(deb > 0) {
        deb--;
        if(deb == 0){
            stable = r;
            bool pressed = (stable == false);
            xQueueSend(qButton, &pressed, 0);
        }
    }
    vTaskDelay(pdMS_TO_TICKS(50));
}

// ----- SHTC3 TASK -----
void vTempHumTask(void *p){
    SHTC3_Init();
    TH_t d;
    bool crc;
    int16_t id;

    while(1){
        xSemaphoreTake(i2cMutex, portMAX_DELAY);
        crc = SHTC3_Measurement(&d.t, &d.h);
        id = SHTC3_Read_Id();
        xSemaphoreGive(i2cMutex);

        if(crc && id != -1) xQueueSend(qTH, &d, 0);
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

// ----- VOC TASK -----
void vVocTask(void *p){
    VOC_t d;
    SGP40_init();
    while(1){
        xSemaphoreTake(i2cMutex, portMAX_DELAY);
        d.voc = SGP40_MeasureVOC(25, 50);
        xSemaphoreGive(i2cMutex);
        xQueueSend(qVOC, &d, 0);
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

// ----- IMU TASK -----
void vImuTask(void *p){
    IMU_t d;
    unsigned t;
    QMI8658_init();
    while(1){
        xSemaphoreTake(i2cMutex, portMAX_DELAY);
        QMI8658_read_xyz(d.acc, d.gyro, &t);
        xSemaphoreGive(i2cMutex);
        xQueueSend(qIMU, &d, 0);
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

```

```

    }
}

// ----- ADC TASKS -----
void vPotTask(void *p){
    ADC_t d;
    while(1){
        adc_select_input(0);
        d.value = adc_read();
        xQueueSend(qPot,&d,0);
        vTaskDelay(pdMS_TO_TICKS(50));
    }
}

void vLightTask(void *p){
    ADC_t d;
    while(1){
        adc_select_input(1);
        d.value = adc_read();
        xQueueSend(qLight,&d,0);
        vTaskDelay(pdMS_TO_TICKS(50));
    }
}

void vSoundTask(void *p){
    ADC_t d;
    while(1){
        adc_select_input(2);
        d.value = adc_read();
        xQueueSend(qSound,&d,0);
        vTaskDelay(pdMS_TO_TICKS(50));
    }
}

// ----- IR TASK -----
void vIrTask(void *p){
    PIO pio = PIO_BLOCK;
    int sm = nec_rx_init(pio, PIN_IR);

    uint32_t raw;
    uint8_t addr, dat;

    while(1){
        while(!pio_sm_is_rx_fifo_empty(pio, sm)){
            raw = pio_sm_get(pio, sm);
            if(nec_decode_frame(raw, &addr, &dat)){
                xQueueSend(qIR, &dat, 0);

                if(dat == 0x45) mode = 0; // приклади
                if(dat == 0x46) mode = 1;
                if(dat == 0x47) mode = 2;
                if(dat == 0x44) mode = 3;
                if(dat == 0x40) mode = 4;
            }
        }
        vTaskDelay(pdMS_TO_TICKS(10));
    }
}

// ----- OLED TASK -----
void vOledTask(void *p){

```

```

uint8_t *img;
uint16_t size = (OLED_1in5_WIDTH/2)*OLED_1in5_HEIGHT;

img = malloc(size);
Paint_NewImage(img, OLED_1in5_WIDTH, OLED_1in5_HEIGHT, 0, WHITE);
Paint_SetScale(16);
Paint_SelectImage(img);

TH_t th={0};
VOC_t vc={0};
IMU_t imu={0};
ADC_t pot={0}, light={0}, sound={0};
bool btn;

while(1){
    // кнопка → переключить дисплей
    if(xQueueReceive(qButton,&btn,0)){
        if(btn) display_on = !display_on;
    }

    if(!display_on){
        OLED_1in5_Clear(0);
        vTaskDelay(pdMS_TO_TICKS(200));
        continue;
    }

    // отримання даних
    xQueueReceive(qTH,&th,0);
    xQueueReceive(qVOC,&vc,0);
    xQueueReceive(qIMU,&imu,0);
    xQueueReceive(qPot,&pot,0);
    xQueueReceive(qLight,&light,0);
    xQueueReceive(qSound,&sound,0);

    Paint_Clear(BLACK);

    char s[32];

    switch(mode){
        case 0:
            Paint_DrawString_EN(5,5,"TEMP/HUM",&Font12,WHITE,BLACK);
            sprintf(s,"T: %.2f",th.t); Paint_DrawString_EN(5,25,s,&Font12,WHITE,BLACK);
            sprintf(s,"H: %.2f",th.h); Paint_DrawString_EN(5,40,s,&Font12,WHITE,BLACK);
            break;

        case 1:
            Paint_DrawString_EN(5,5,"AIR",&Font12,WHITE,BLACK);
            sprintf(s,"VOC: %u",vc.voc);
            Paint_DrawString_EN(5,25,s,&Font12,WHITE,BLACK);
            break;

        case 2:
            Paint_DrawString_EN(5,5,"IMU",&Font12,WHITE,BLACK);
            sprintf(s,"AX %.2f",imu.acc[0]); Paint_DrawString_EN(5,25,s,&Font12,WHITE,BLACK);
            sprintf(s,"GY %.2f",imu.gyro[1]); Paint_DrawString_EN(5,40,s,&Font12,WHITE,BLACK);
            break;

        case 3:
            Paint_DrawString_EN(5,5,"ADC",&Font12,WHITE,BLACK);

```

```

        sprintf(s,"Pot %u",pot.value); Paint_DrawString_EN(5,25,s,&Font12,WHITE,BLACK);
        sprintf(s,"Light %u",light.value); Paint_DrawString_EN(5,40,s,&Font12,WHITE,BLACK);
        sprintf(s,"Sound %u",sound.value); Paint_DrawString_EN(5,55,s,&Font12,WHITE,BLACK);
        break;

    case 4:
        Paint_DrawString_EN(5,5,"ALL",&Font12,WHITE,BLACK);
        sprintf(s,"T %.1f H %.1f",th.t,th.h);
        Paint_DrawString_EN(5,25,s,&Font12,WHITE,BLACK);
        sprintf(s,"VOC %u",vc.voc);
        Paint_DrawString_EN(5,40,s,&Font12,WHITE,BLACK);
        break;
    }

    OLED_1in5_Display(img);
    vTaskDelay(pdMS_TO_TICKS(250));
}

// ----- MAIN -----
int main(){
    stdio_init_all();
    DEV_Module_Init();
    OLED_1in5_Init();
    OLED_1in5_Clear(0);

    adc_init();
    adc_gpio_init(26);
    adc_gpio_init(27);
    adc_gpio_init(28);

    i2cMutex = xSemaphoreCreateMutex();

    qButton = xQueueCreate(5,sizeof(bool));
    qTH     = xQueueCreate(5,sizeof(TH_t));
    qVOC    = xQueueCreate(5,sizeof(VOC_t));
    qIMU    = xQueueCreate(5,sizeof(IMU_t));
    qPot    = xQueueCreate(5,sizeof(ADC_t));
    qLight  = xQueueCreate(5,sizeof(ADC_t));
    qSound  = xQueueCreate(5,sizeof(ADC_t));
    qIR     = xQueueCreate(5,sizeof(uint8_t));

    xTaskCreate(vButtonTask,"btn",512,NULL,2,NULL);
    xTaskCreate(vTempHumTask,"th",512,NULL,2,NULL);
    xTaskCreate(vVocTask,"voc",512,NULL,2,NULL);
    xTaskCreate(vImuTask,"imu",512,NULL,2,NULL);

    xTaskCreate(vPotTask,"pot",512,NULL,2,NULL);
    xTaskCreate(vLightTask,"light",512,NULL,2,NULL);
    xTaskCreate(vSoundTask,"sound",512,NULL,2,NULL);

    xTaskCreate(vIrTask,"ir",512,NULL,3,NULL);
    xTaskCreate(vOledTask,"oled",1024,NULL,1,NULL);

    vTaskStartScheduler();
    while(1);
}

```

Висновок: У роботі було створено повноцінну багатозадачну систему FreeRTOS із підтримкою ІЧ-пульта за протоколом NEC, сенсорів, аналогових датчиків та OLED-дисплея. Дані від усіх задач передавалися через черги, а доступ до І²C захищався м'ютексом, що забезпечує коректну взаємодію сенсорів. Дисплей отримував дані з різних джерел та перемикав режими роботи відповідно до команд пульта. Система працює в реальному часі та адекватно реагує на дії користувача, забезпечуючи стабільне та передбачуване керування.